SYSTEM REQUIREMENTS SPECIFICATION / VERSION 1.0

Clinic Network Information Portal

54-branch read-only integration architecture and prototype delivery specification

Prepared for: Clinic stakeholders and implementation team

Document date: 03 November 2026

Status: Proposed architecture and prototype requirements

Repository: clinic-network-portal

Companion implementation contract: README.md

Document status

This is a comprehensive proposed SRS, not an as-built system description. The existing application, local database engine, schema, workflows, and regulatory deployment constraints have not yet been inspected. All candidate clinical modules and target performance values require stakeholder review.

01 Executive summary

The clinic operates 54 independent branches. Each branch has its own locally hosted database, reportedly using an identical schema. Headquarters requires remote, read-only access through a single responsive web portal. The portal must provide detailed module pages and source-linked records, not only dashboards. The original clinical application must remain unchanged.

The recommended target is a central read model populated through one configurable, locally deployed branch connector per branch. The connector retrieves authorized data with least-privilege read access and sends it over authenticated outbound HTTPS to a central ingestion service. A central Express API exposes approved, branch-aware information to the React web portal.

The first milestone is a fully navigable React + TypeScript prototype, seeded with synthetic data and designed around API-ready repository interfaces. No live clinic data, central backend, or connector is implemented in this milestone.

02 Document control and requirements authority

Item	Position
Product	Clinic Network Information Portal
Initial rollout	Frontend prototype with fictional mock data
Production integration	Proposed; pending discovery and feasibility testing
Clinical source system	Existing application remains unmodified
Data access	Read-only source access; central portal permissions are role and branch scoped
Requirement precedence	Verified legacy features and approved stakeholder decisions supersede candidate examples
Change control	Review scope additions with product owner, clinical lead, privacy/security lead and integration lead

2.1 Confirmed decisions

- Fifty-four distinct branch databases; same structure reported, not independently verified.
- Headquarters and other authorized roles require internet-accessible detailed read-only information.
- No source application code modifications and no source write-back.
- Responsive React prototype first, realistic mock data, backend integration later.
- Modern, distinctive, professional design; avoid excessive popups and template-like dashboards.

2.2 Not yet confirmed

Exact database product/version; OS/hardware at branches; existing module inventory; full relational schema; which business rules are implemented outside the database; file storage; retention periods; legal authorization for each sensitive field; allowable hosting geography; synchronization interval and load; support model; existing vendor APIs/exports.

03 Stakeholders and actors

Stakeholder	Responsibilities
Executive sponsor	Approves business outcomes, budget, priority and rollout.
Clinic operations owner	Validates branch scope, work practices and reporting needs.
Clinical governance representative	Approves permitted clinical data visibility and meaning.
Privacy/security lead	Approves access policy, retention, threat controls and privacy review.
Legacy vendor / DBA	Confirms DB engine/schema, safe extraction method and supported interfaces.
Implementation team	Builds React prototype, integration, central services and tests.
Operations / IT	Deploys connectors, manages credentials, monitoring, incident response.

04 Scope and exclusions

4.1 In scope for target system

• Responsive, authenticated portal with organization and authorized branch scope.

• Read-only list, search, filter, drill-down and related-record navigation for each approved legacy module.

• Branch directory, branch status, synchronization freshness and operational reporting.

• Centralized branch-aware storage, query services, secure connector onboarding and health monitoring.

• Access control by role, module and branch; access logging and controlled exports where approved.

• Resilient synchronization of approved inserts, changes and deletions or logical removals.

4.2 Excluded or deferred

• Writing records back into local clinical databases; performing source-clinic approvals or operational transactions.

• Replacing the existing application or changing its workflow, UI, schema or code.

• Automatic cross-branch patient identity merging without an approved master-patient strategy.

• Real production patient data during prototype development.

• Live claims of data accuracy/freshness in a mock-data demonstration.

05 Proposed product information architecture

The portal must be a read-only counterpart of approved source-system functionality. Since no verified legacy menu or schema has been supplied, the following are candidate modules, not a claim that the existing clinic software supports them.

Candidate area	List / read-only detail examples
Workspace and branches	Organization overview, 54-branch directory, branch profile, branch context switch
Patients	Patient directory, patient detail, linked source visits and permitted history
Visits / encounters	Visits list, encounter details, practitioner context, chronology
Appointments	Appointment list/calendar view, appointment detail and status
Laboratory	Lab requests, result details and source-linked visit
Pharmacy	Dispensing records, medication detail; inventory only if verified
Billing / finance	Invoices, payments, line items, status and traceable totals
Staff / administration	Staff directory and source-linked branch assignment
Reports	Permitted aggregate reports, filters, authorized exports
Integration health	Connector status, last success, lag, error history
Access / audit	Role permissions, read-only access log, denial and activity states

Source fidelity gate

A page can be approved for production only after fields, data relationships, calculations and semantics have been checked against the actual legacy application and database. A missing source field is not an invitation to invent a replacement clinical value.

06 Roles, scopes and access behavior

The following roles are illustrative. Production permission assignments must be explicitly approved and enforced server-side. Organization-wide scope does not automatically imply unrestricted access to sensitive records.

Role	Illustrative branch scope	Illustrative access
HQ administrator	All 54	Approved portal oversight and integration administration
Executive manager	All 54	Approved operational/financial summaries and details
Regional manager	Assigned branches	Only approved modules and assigned branches
Branch manager	One assigned branch	Permitted branch records and reports
Finance officer	Assigned branches	Finance and approved reports only
Clinical supervisor	Assigned branches	Specifically approved clinical views

- Access is the intersection of role, module permission, branch scope and record-level policy.
- Enforce all four restrictions in the backend/API, not only in React menus.
- Deep links, search results, exports and API endpoints must respect the same restrictions.
- Audit sensitive record views and export actions. Default-deny clinical fields until formally approved.

07 Functional requirements register

Identifiers below support design reviews, testing and change tracking. All functionality depends on verified availability of corresponding source data.

ID	Requirement
FR-01	The portal shall display an authorized, searchable 54-branch directory with branch profile and status.
FR-02	The portal shall preserve branch context in views, navigation, and record identity.
FR-03	The portal shall provide list and drill-down details for every approved legacy module.
FR-04	The portal shall support search, filters, sort and pagination in substantial record lists.
FR-05	The portal shall provide contextual links between related records where source relations are verified.
FR-06	All source clinical/financial record operations in the portal shall be read-only.
FR-07	The API shall enforce module-, branch-, and record-scoped authorization.
FR-08	The portal shall clearly display source branch and synchronization freshness for synced data.
FR-09	The integration shall support bounded initial backfill, incremental transfer and correction/deletion handling.
FR-10	The integration shall provide durable acknowledgment, retry, checkpoints and idempotent processing.

ID	Requirement
FR-11	The platform shall show health/error status for each of the 54 branch connectors.
FR-12	Authorized aggregate reports shall retain branch provenance and disclose incomplete/stale coverage.
FR-13	The platform shall log security-relevant data access, connector changes and export activity.
FR-14	The portal shall show permission denied, data unavailable, stale, empty and partial-sync states.
FR-15	The prototype shall supply a deterministic mock repository, mock roles and synthetic sample records.
FR-16	The prototype shall operate through domain repository interfaces that can later be fulfilled by HTTP adapters.
FR-17	The portal shall provide responsive, keyboard-accessible interaction at desktop, tablet and mobile sizes.
FR-18	If documents/images are in scope, the integration shall preserve approved metadata and retrieve assets through an authorized method.

08 System architecture and responsibilities

Branch tier: unchanged clinic app and DB; independent read-only connector with persistent local state. Central ingestion tier: authenticated HTTPS API, validation, schema mapping, durable ingestion and observability. Central data tier: branch-indexed PostgreSQL read store, protected backups and retention. Presentation tier: Express read API with enforced access control and React web portal.

Tier	Proposed technology	Boundary
Branch agent	C# + .NET Worker Service (conditional)	Windows service only after branch OS is confirmed
Source adapter	DB-specific driver/queries	Read-only access; supports verified extraction method
Transport	TLS-protected HTTPS 443	Outbound from branch to authenticated central endpoint
Ingestion	Node.js + Express + TypeScript	Connector authentication, validation and acknowledgement
Read store	PostgreSQL	Private database; branch-aware source identities
Read API	Express + TypeScript	Authenticated user requests and authorized projections
UI	React + TypeScript + Vite	Responsive read-only client application
Monitoring	Metrics, logs, alerts; tool TBD	Per-branch health and ingestion lag

8.1 Why a connector does not publish the local database

The connector is an outbound HTTPS client. Its installation does not make the source DB or branch workstation a public server. Branch routers generally require no inbound port forwarding; the centrally hosted endpoint receives uploads. Headquarters queries only the central API and read store. Firewall rules and proxies must still permit the authorized outbound destination.

8.2 Alternative designs considered

Option	Assessment
Custom connector + central read store	Preferred preliminary design for detailed asynchronous read access and independent branches.
Vendor-supported integration/export	Potentially better where certified and sufficiently complete; examine before writing a connector.
Direct live branch queries over private VPN	Could deliver fresher data, but couples browsing to 54 live networks, availability and permission boundaries.
Third-party CDC/replication platform	Possible for supported engines; operational footprint and privileges may be inappropriate for branch sites.

09 Connector executable: detailed operating specification

A lightweight connector is a small, independent service with one purpose: safely extract approved changes from a source system and deliver them to the central ingestion endpoint. “Lightweight” does not imply zero resource cost; measured CPU, disk, RAM and query load determine acceptable operation.

9.1 Proposed packaging and deployment

• Build one reusable agent in C#/.NET, then produce a versioned, signed Windows installer and service executable if Windows is confirmed.

• Install on an authorized machine with safe network connectivity to the local clinic DB; it need not be the receptionist workstation.

• Configure per-branch identity, allowed source, approved extraction profile, server endpoint and operational limits.

• Protect DB credentials and connector secrets using OS-supported secret storage or an approved secure alternative.

• Start automatically at boot, recover from restarts, and expose local diagnostic logs with no patient payload leakage.

• Deploy and roll back versions using controlled approvals and a branch inventory.

9.2 Connector execution loop

Step	Operation
01	Load validated configuration; verify service identity and source connectivity.
02	Read the durable extraction checkpoint, using source-specific cursor/version semantics.
03	Extract a bounded snapshot or change window with least-privilege read operations.
04	Transform only approved fields; preserve source identifiers, branch, timestamps and operation type.
05	Construct deterministic batch IDs and integrity metadata; persist any required retry state.
06	Send batch to central API over authenticated TLS; retry transient failures with backoff.
07	Server validates, authorizes and durably applies or queues the batch; returns precise acknowledgement.
08	Advance the checkpoint only for acknowledged content; expose metrics and repeat.

9.3 Change tracking and deletion strategy

Preferred extraction mode depends on source DB capabilities. Evaluate database-native CDC/change tracking or supported vendor export first. If reliable update timestamps and stable keys exist, controlled keyset/time-window polling plus reconciliation may work. A simple WHERE updated_at > last_timestamp is not automatically safe due to timestamp ties, late commits, deletes and source corrections. If deletes cannot be observed, schedule reconciliation and define how long stale records may remain central.

9.4 Loss, duplication and ordering defenses

• Every record has a stable composite key: branch_id + entity_type + source_record_id.

• Central ingest is idempotent across batch resends and protects against stale/out-of-order updates.

• Local checkpoints advance only after durable central acknowledgement; retries remain safe after crash or network failure.

• Initial historical backfill uses page/batch limits and a consistent snapshot strategy where supported.

• Backpressure prevents overwhelming the branch DB or central server; failed branches remain visible in monitoring.

• Reconciliation compares counts, hashes or selected source slices and flags unexplained mismatches.

10 Illustrative end-to-end workflows

WF-01: Patient record created at Branch 12

1. A worker uses the unchanged legacy app. 2. Its local database commits a patient record. 3. The connector identifies the insert through the verified change method. 4. The connector prepares an approved, branch-bound payload. 5. The central ingestion endpoint authenticates the connector and processes the event idempotently. 6. A source-identified central patient record is updated. 7. An authorized HQ user opens Patients > Branch 12 > Patient detail and sees the record after sync, with freshness provenance.

WF-02: Manager browses a multi-page read-only journey

1. Manager signs in to the central portal. 2. Backend resolves approved branch and module scope. 3. Manager opens the branch directory and selects Branch 12. 4. Manager searches patients, opens a profile, follows a permitted encounter, and inspects a related laboratory result. 5. Each request is checked against the same permissions. 6. The portal shows the source branch and relevant sync watermark, and permits no source writes.

WF-03: Connectivity interruption

1. Branch loses internet. 2. Legacy clinic operation continues locally. 3. Connector cannot deliver and preserves its checkpoint/retry state. 4. Central system marks Branch 12 delayed/offline based on observed heartbeat policy. 5. HQ sees existing synced data as stale, not live. 6. When connection returns, the connector resumes and reconciliation checks parity.

WF-04: Existing source record corrected or removed

1. A source record changes or is removed. 2. Supported change tracking or reconciliation identifies the event. 3. Central store updates its read model, preserving appropriate audit provenance. 4. Reports are recomputed from synchronized current data. 5. If delete detection is impossible, the portal must not claim deletion fidelity until a validated compensating process exists.

WF-05: New branch onboarding

1. IT approves installation and verifies DB and OS compatibility. 2. Central platform provisions a unique connector identity bound to the branch. 3. IT installs and configures agent securely. 4. Agent performs connection/self-test and controlled initial backfill. 5. Source/target parity and resource impact are validated. 6. Branch becomes visible as production-connected only after signoff.

11 Data model, mapping and provenance

The central model must preserve source identity and distinguish locally repeated IDs. Example: patient 1001 from BR-001 is different from patient 1001 from BR-054 unless a separate, approved matching process establishes an association.

Entity / record	Minimum conceptual fields
Branch	branch_id, display_name, geographic/organizational scope, connector state, last_sync
Source reference	branch_id, entity_type, source_record_id, source_version, synced_at

Entity / record	Minimum conceptual fields
Patient / visit	Permitted clinical fields, source reference, linked branch-scoped entities
Invoice / payment	Permitted amounts/currency/status, source relationships and sync provenance
Connector checkpoint	branch_id, stream/table, extraction cursor, last ack, agent version
Sync event / batch	batch_id, branch_id, event type, delivery status, counts, error code
Audit event	actor, action, authorized scope, timestamp, data category, outcome

Storage decisions: only approved data categories are replicated; encrypted-at-rest central storage; retention and deletion policy; partition/index strategy based on measured volume; consistent currency/timezone presentation; source raw identifiers retained in protected internal fields. Source file-system assets need a separate authorized ingestion path, not assumed access.

12 API boundaries and sample contracts

API routes are proposed integration contracts, not existing endpoints. The connector authenticates as a machine principal, while managers authenticate as user principals. These credential types and permission scopes must remain separate.

API boundary	Representative route / behavior
Connector ingestion	POST /api/v1/sync/batches - validate source binding, process batch, acknowledge
Connector heartbeat	POST /api/v1/connectors/heartbeat - connectivity/version status
Branch read	GET /api/v1/branches - filtered to authorized scope
Patient read	GET /api/v1/patients?branchId=BR-012 - paginated, scoped
Patient detail	GET /api/v1/patients/BR-012:1001 - branch-qualified identity
Sync health	GET /api/v1/sync-health - authorized operational status
Reports	GET /api/v1/reports?... - authorized derived aggregates

Standard read responses should distinguish `items`, `page`, `pageSize`, `total`, `asOf`, `coverage`, and `warnings` where relevant. Errors should differentiate 401 unauthenticated, 403 forbidden, 404 scoped missing, 409 sync conflict and service unavailable. Never include raw medical payloads in operational logs.

13 Prototype engineering contract

The README.md is the Codex-facing prototype implementation contract. The initial build must use React + TypeScript + Vite, responsive bespoke design tokens, router-driven detailed pages, deterministic synthetic mock data, and typed asynchronous read-repository interfaces. The production connector and APIs are architectural targets, not items to build during this phase.

13.1 Required prototype behaviors

• 54 branches with plausible fictional statuses, one selected organization/branch context, and deep links.

• At least representative list/detail journeys for patients, visits, lab, pharmacy, billing and other approved candidate areas.

• Mock roles and module/branch restrictions applied inside the mock repository and route guards.

• Loading, empty, permission-denied, stale, offline and error states with coherent presentation.

• Search, sort, filter, pagination and CSV demo export limited to synthetic authorized records.

• At-a-glance organization and branch overview metrics derived from the actual mock records.

• Responsive layouts validated at 360, 390, 768, 1024, 1280, 1440, 1920 pixels.

• No operative create/edit/delete controls, production login claims, real patient data or fake online connection.

13.2 UX character

Purpose-built healthcare operations visual language: neutral/natural surfaces, disciplined typography, consistent tabular data, purposeful color, compact navigation, considered detail composition, limited modals, and human-readable data density. Avoid repeated card-grid dashboards, decorative gradients, bubbly UI, emoji, and Lucide React icons. Prefer professional SVG icons such as Phosphor and accessible interaction primitives.

13.3 Mock-to-API swap

Pages call typed domain services, never import seed arrays. `ClinicReadRepository` exposes asynchronous list/detail/report operations with pagination and filters. An eventual HTTP implementation replaces the mock adapter without rewriting page components. Composite record identity and scope filtering are modeled in the prototype.

14 Non-functional requirements

ID	Requirement / target
NFR-01	Security: authenticated endpoints, least privilege, branch-scoped authorization, TLS, secure key storage, MFA for production users, detailed audits.
NFR-02	Privacy: data minimization, purpose limitation, approved sensitive-data access, retention and jurisdictional legal review.
NFR-03	Reliability: recover after branch restarts/network outages without silent data loss; idempotent ingestion and durable checkpoints.
NFR-04	Data correctness: traceable source mappings, reconciliation, updates and deletion handling; no unapproved patient merges.
NFR-05	Freshness: user-visible per-branch last-sync and stale thresholds. Exact sync target to be agreed after measurements.
NFR-06	Availability: HQ can browse already synchronized records during branch outage; central service availability SLO to be negotiated.
NFR-07	Performance: bounded source queries; paginated APIs; target response times/load limits measured and approved in PoC.
NFR-08	Scalability: capacity for 54 agents and projected records; load test ingestion and common read workflows.
NFR-09	Accessibility: keyboard operability, contrast, labels, focus, responsive use on 360px and larger viewports.
NFR-10	Maintainability: versioned agent and schema mappings, CI tests, staged rollout, rollback, documentation and monitoring.
NFR-11	Disaster recovery: encrypted backups, restore tests, agreed RPO/RTO and clear source-of-truth designation.
NFR-12	Observability: per-branch heartbeat, lag, transfer counts, failures, central ingestion audit and alert escalation.

Do not promise numeric sync intervals, uptime, RPO/RTO, branch CPU usage or maximum query latency until the clinic provides requirements and testing supplies evidence.

15 Security, privacy and regulatory review

This is a healthcare information system dealing potentially with highly sensitive personal data. Before deployment, qualified stakeholders must confirm applicable Ethiopian personal-data protection and healthcare obligations, appropriate legal basis, access purpose, retention and cross-border data-handling constraints. This document does not substitute for a legal or clinical privacy assessment.

• Never expose branch database listener ports publicly; use outbound HTTPS and hardened central services.

• Use dedicated read-only source DB identity, restricted to required tables/views and connection sources.

• Authenticate each agent with a unique machine credential; bind the authenticated identity to its registered branch.

• Central API enforces authorization independently of UI, including exports and related-record navigation.

• Encrypt transport and stored data; protect backups and secrets; minimize sensitive information in logs.

• Support incident response, revocation of compromised connectors, audit retention and periodic access review.

• Assess public-cloud versus in-country/private hosting against actual legal and institutional requirements.

• Use only fictional data in the prototype and sanitized test data in proof-of-concept environments.

16 Verification and acceptance

Test ID	Acceptance scenario / evidence
AT-01	A mock HQ user browses all 54 branch cards/rows and selects a branch.
AT-02	User navigates patient list to detail to associated encounter to lab record with correct branch context.
AT-03	Finance role can open permitted financial pages and is denied protected clinical details.
AT-04	Regional role cannot retrieve records of a disallowed branch through direct URL or repository calls.
AT-05	Two branches can hold local source ID 1001 with no identity collision.
AT-06	Offline branch is labeled stale and its last successful sync is visible.
AT-07	Filters, pagination, search, breadcrumb navigation and browser refresh preserve coherent state.
AT-08	UI has no page overflow at 360/390px, and keyboard navigation is practical.
AT-09	Metrics reconcile with the displayed synthetic records; no disconnected random totals.
AT-10	One-branch connector PoC matches sampled source records and tolerates duplicate batch delivery.
AT-11	Connector resumes after internet failure/crash without losing acknowledged or unacknowledged changes.
AT-12	Source load during backfill/incremental extraction remains within approved DB operational limits.

17 Delivery plan and governance gates

Gate	Activities and exit criterion
G0 Discovery	Inventory legacy pages/menus, DB engine/schema, business rules, access policy, network, legal constraints; get scope signoff.
G1 Frontend prototype	Demonstrate full candidate information architecture using typed synthetic repository; responsive and role scoped.
G2 One-branch integration PoC	Verify read account, extraction algorithm, backfill, updates/deletes, outages and parity with source.
G3 Production readiness	Security/privacy reviews, capacity, backup/restore, monitoring, incident procedures, agent installer/rollback.
G4 Controlled rollout	Pilot several branches; compare data and operational impact; progressively onboard up to 54.
G5 Operations handoff	Runbooks, ownership, patch process, monitoring alerts, ongoing reconciliation and support.

18 Risk register

Risk	Impact	Mitigation
Unsupported DB engine or inaccessible source	Critical	Vendor/DBA discovery before connector commitment
Legacy business logic not visible in schema	High	Validate each displayed field against legacy UI
Missing deletes or unreliable timestamps	High	Use supported CDC or reconciliation; limit claims
Branch hardware/network weak	High	Bounded extraction, backoff and staged rollout
Sensitive data copied too broadly	Critical	Data minimization, role policy, legal/privacy signoff
Cross-branch ID collision	High	Composite source identity, no implicit patient merge
Sync drift / incorrect totals	High	Parity controls, provenance and anomaly alerts
Agent compromise or credential exposure	Critical	Unique machine identities, revocation, secure secrets
Prototype mistaken for deployed system	Medium	Persistent demo labels and no live-data claims
Unapproved scope expansion	High	Legacy inventory and stakeholder-controlled change process

19 Open questions for the next technical meeting

• Which RDBMS and version does the existing clinic application use, and is there an approved supported API/export feature?

• Which 54 branch computers/servers host the DB, and which operating systems are they running?

• Can IT issue a dedicated least-privilege read-only DB account without modifying application code?

• What are all actual legacy menus, fields, data relationships, reports and user roles?

• Does the database have native CDC/change tracking, stable change fields or deletions/tombstones?

• Are scanned documents, images and attached files in the DB, on disk or in another service?

• How many rows/GB exist per branch and how much changes per day?

• What refresh delay is clinically/operationally acceptable, and which views need especially fresh data?

• Who is legally authorized to view patient details across branches, and what must be masked?

• Where may sensitive information be hosted, what retention applies and who approves exports?

• Who owns branch agent installation, updates, support and incident response?

• What cloud/private infrastructure, expected user count, budget and availability goals apply?

20 Decision and approval record

Decision	Current position
GitHub repository	clinic-network-portal
Local folder	clinic-connector (can remain unchanged)
First milestone	React + TypeScript, deterministic mock data, API-ready read repositories
Source application	Unmodified; read-only data integration
Production connector	Proposed custom C#/.NET service if infrastructure supports it
Central stack	Proposed Express/Node.js + PostgreSQL
Sync mechanism	To be selected after source DB inspection
UI modules	Candidate inventory until legacy screens and schema are validated
Compliance/hosting	Formal review required before processing live patient data

Companion specification: README.md. Treat the README as the prototype work contract and this PDF as the full system-level architecture/SRS. Any unverified technical assumption must be validated before production commitments.